

Лабораторная работа №4

Моделирование конфликта Защитник–Нападающий

Бабенко Артём Сергеевич

Содержание

1	Цель работы	4
2	Задание	5
3	Теоретическое введение	6
4	Выполнение лабораторной работы	7
4.1	Создание проекта	7
4.2	Основной модуль	8
4.3	Запуск симуляций	9
4.4	Построение графиков	10
4.5	Дополнительное задание	12
4.6	Генерация производных форматов	13
4.7	Реализация модуля simulation.jl	14
4.8	Запуск симуляций	19
4.9	Визуализация результатов	19
4.10	Дополнительное задание	20
5	Контрольные вопросы	23
6	Выводы	25
	Список литературы	26

Список иллюстраций

4.1	Инициализация проекта	7
4.2	Установка пакетов с помощью скрипта	7
4.3	Скрипт основного модуля	9
4.4	Скрипт запуска симуляций	10
4.5	Скрипт построения графиков	11
4.6	Зависимость вероятности атаки на первый актив p_1 от отношения ценностей V_1/V_2	11
4.7	Средний выигрыш Нападающего	12
4.8	Скрипт дополнительного задания	13
4.9	Сравнение платёжных матриц	13
4.10	Запуск генерации производных форматов	14
4.11	Полученные файлы других форматов	14

1 Цель работы

Освоить применение теории игр для анализа противостояния в сфере информационной безопасности. Научиться строить матричную игру, находить равновесие Нэша в чистых и смешанных стратегиях.

2 Задание

- Формализовать конфликт «Защитник–Нападающий» в виде антагонистической игры.
- Реализовать на Julia функции расчёта платёжной матрицы, поиска равновесия Нэша и симуляции игры.
- Визуализировать зависимость ожидаемого выигрыша и равновесных вероятностей от стоимости защиты и величины ущерба.

3 Теоретическое введение

Теория игр — раздел математики, изучающий модели принятия решений в условиях конфликта. В информационной безопасности она применяется для анализа стратегического взаимодействия Защитника (Defender) и Нападающего (Attacker).

Основные элементы модели: - **Игроки**: Защитник и Нападающий. - **Стратегии**: доступные каждому игроку действия (защитить или атаковать конкретный узел сети). - **Функции выигрыша**: количественная оценка результата для каждого игрока в зависимости от выбранных стратегий.

Антагонистическая игра — игра с прямо противоположными интересами игроков: выигрыш одного равен проигрышу другого. Модель «Защитник–Нападающий» является игрой с нулевой суммой, так как сумма выигрышей постоянна: - При успешной атаке: $(V_i - c_a) + (-V_i - c_d) = -c_a - c_d$ - При отражённой атаке: $(-c_a) + (-c_d) = -c_a - c_d$

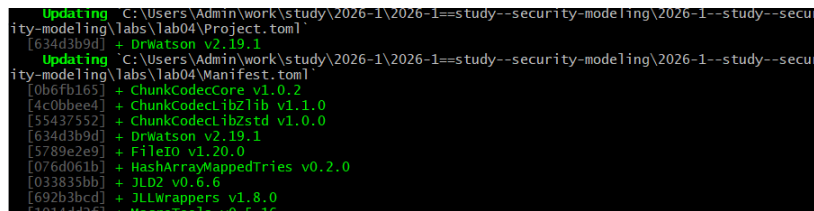
Выигрыш Нападающего — это ущерб Защитника, что отражает антагонизм их интересов.

Для анализа стратегического взаимодействия применяется теория игр [2]. Реализация модели выполнена на языке Julia с использованием пакетов DrWatson [1] и Literate [3].

4 Выполнение лабораторной работы

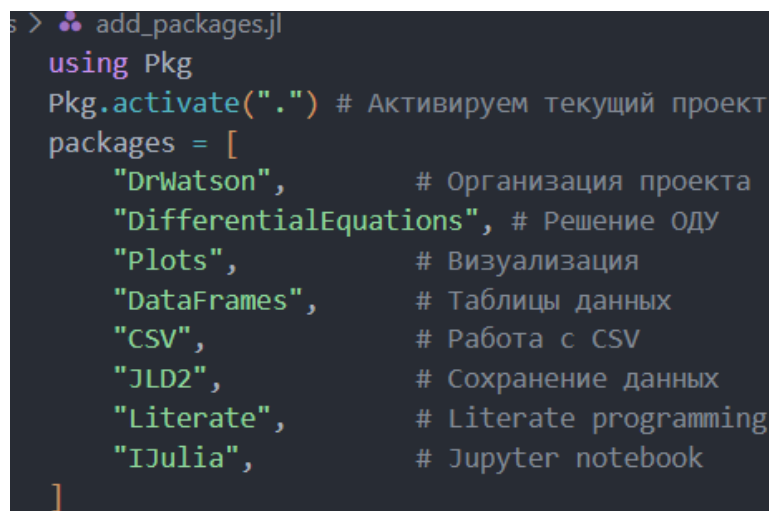
4.1 Создание проекта

Создадим проект с помощью DrWatson и установим необходимые пакеты для дальнейшей работы с помощью скрипта `add_packages.jl` (рис. 4.1, рис. 4.2).



```
Updating C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--secu
ity-modeling\labs\lab04\Project.toml
[634d3b9d] + DrWatson v2.19.1
Updating C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--secu
ity-modeling\labs\lab04\Manifest.toml
[0b6fb165] + ChunkCodecCore v1.0.2
[4c0bbe4] + ChunkCodecLibZlib v1.1.0
[55437552] + ChunkCodecLibZstd v1.0.0
[634d3b9d] + DrWatson v2.19.1
[5789e2e9] + FileIO v1.20.0
[076d061b] + HashArrayMappedTries v0.2.0
[033835bb] + JLD2 v0.6.6
[692b3bcd] + JLLWrappers v1.8.0
[1012d225] + MbedTLS v0.5.16
```

Рисунок 4.1: Инициализация проекта



```
s > add_packages.jl
using Pkg
Pkg.activate(".") # Активируем текущий проект
packages = [
    "DrWatson",          # Организация проекта
    "DifferentialEquations", # Решение ОДУ
    "Plots",              # Визуализация
    "DataFrames",         # Таблицы данных
    "CSV",                 # Работа с CSV
    "JLD2",                # Сохранение данных
    "Literate",           # Literate programming
    "IJulia",              # Jupyter notebook
]
```

Рисунок 4.2: Установка пакетов с помощью скрипта

4.2 Основной модуль

Файл `src/simulation.jl` является основным модулем, используемым для моделирования (рис. 4.3). Он содержит несколько функций:

- `build_payoff_matrices(V::Vector{Float64}, c_a::Float64, c_d::Float64)` — строит две платёжные матрицы игры размера $n \times n$.
- `mixed_nash_2x2(A::Matrix{Float64}, D::Matrix{Float64})` — ищет равновесие Нэша для биматричной игры 2×2 .
- `run_simulation(params::Dict)` — проводит один эксперимент с заданными параметрами игры.
- `generate_params()` — формирует полную сетку параметров для симуляционного эксперимента.
- `main_simulations()` — основная функция запуска всех симуляций.
- `load_results()` — загружает ранее сохранённые результаты из CSV-файла.


```

src > simulation.jl
1 using LinearAlgebra
2 using DataFrames, CSV
3 using DrWatson
4
5 # Строит платёжные матрицы A (атакующий) и D (защитник)
6 # для n объектов с ценностями V, стоимостями атаки c_a и защиты c_d.
7 # i – узел атаки, j – узел защиты:
8 # если i != j: атакующий получает V[i]-c_a, защитник теряет V[i]+c_d
9 # если i == j: атака отражена, оба несут только затраты
10 function build_payoff_matrices(v::Vector{Float64}, c_a::Float64, c_d::Float64)
11     n = length(v)
12     A = zeros(n, n)
13     D = zeros(n, n)
14     for i = 1:n, j = 1:n
15         if i != j
16             A[i, j] = v[i] - c_a
17             D[i, j] = -v[i] - c_d
18         else
19             A[i, j] = -c_a
20             D[i, j] = -c_d
21         end
22     end
23     return A, D
24 end
25
26 # Находит равновесие Нэша в игре 2x2.
27 # Сначала проверяет чистые стратегии (седловая точка),
28 # затем вычисляет смешанные через условие безразличия.
29 function mixed_nash_2x2(A::Matrix{Float64}, D::Matrix{Float64})
30     for i = 1:2, j = 1:2
31         if A[i, j] >= A[3-i, j] && D[i, j] >= D[i, 3-j]
32             p = zeros(2); p[i] = 1.0
33             q = zeros(2); q[j] = 1.0
34         end
35     end
36 end

```

Рисунок 4.3: Скрипт основного модуля

4.3 Запуск симуляций

Скрипт `scripts/run_sims.jl` печатает сообщение о запуске, выполняет `main_simulations()` (все комбинации параметров) и выводит количество рассчитанных вариантов (рис. 4.4).

```
ts > run_sims.jl
▷Run | ▷Above | ▷Below | ▷Debug
# # Запуск симуляций

using DrWatson
@quickactivate "project"

include(scrdir("simulation.jl"))

▷Run | ▷Above | ▷Below | ▷Debug
# ## Запуск

println("Запуск симуляций...")
results = main_simulations()
println("Готово! Сохранено строк: ", nrow(results))
```

Рисунок 4.4: Скрипт запуска симуляций

4.4 Построение графиков

Файл `scripts/plot_results.jl` выполняет следующие действия (рис. 4.5):

- Загружает результаты через `load_results()`.
- Фильтрует строки с `c_a = 1.0` и `c_d = 1.0`.
- Строит точечный график зависимости вероятности атаки на первый актив `p1` от отношения ценностей `V1/V2` (рис. 4.6). Точки разделены по типу равновесия (`pure/mixed`). Сохраняет в `plots/p1_vs_ratio.png`.
- Группирует отфильтрованные данные по `V1` и `V2`, вычисляет средний ожидаемый выигрыш Нападающего `UA_mean`.
- Строит тепловую карту среднего выигрыша Нападающего в координатах `(V1, V2)` (рис. 4.7). Сохраняет в `plots/heatmap_UA.png`.

```

julia> plot_results.jl
└─ Run | Above | Below | Debug
# # Визуализация результатов

using DrWatson
@quickactivate "project"

using Plots, DataFrames, Statistics

include(sourcedir("simulation.jl"))

└─ Run | Above | Below | Debug
# ## Загрузка данных

results = load_results()

└─ Run | Above | Below | Debug
# ## Фильтрация: c_a = 1, c_d = 1

filtered = results[(results.c_a .== 1.0) .& (results.c_d .== 1.0), :]
ratio = filtered.V1 ./ filtered.V2
scatter( ratio,
         filtered.p_1,
         group = filtered.type,
         xlabel = "V1 / V2",
         ylabel = "p1 (вероятность атаки на актив 1)",
         title = "Стратегия Нападающего (c_a=1, c_d=1)",

```

Рисунок 4.5: Скрипт построения графиков

Запускаем этот скрипт и в результате выполнения `scripts/plot_results.jl` получаем следующие графики (рис. 4.6, рис. 4.7):

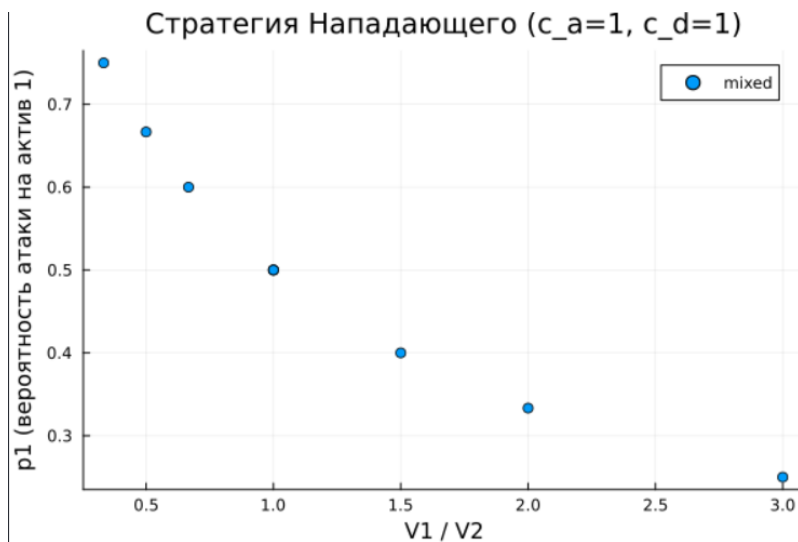


Рисунок 4.6: Зависимость вероятности атаки на первый актив p_1 от отношения ценностей V_1/V_2

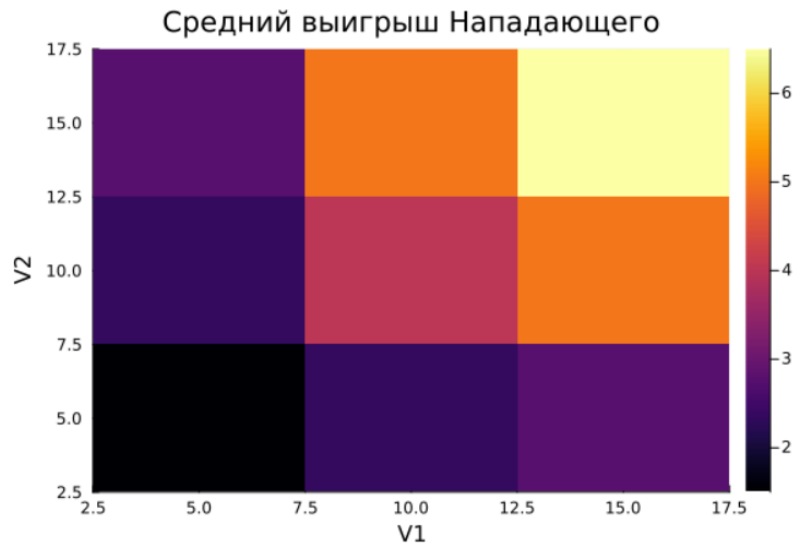


Рисунок 4.7: Средний выигрыш Нападающего

4.5 Дополнительное задание

В рамках дополнительного задания рассмотрим расширенную модель 3×3 , добавляющую стратегии «не атаковать» для Нападающего и «не защищать ничего» для Защитника (рис. 4.8).

```

pts > extra_tasks.jl

ENV["GKSwstype"] = "100"

using DrWatson
@quickactivate "project"

>Run | ⬆Above | ⬆Below | ⚙Debug
# ## Загрузка зависимостей

using Plots, LinearAlgebra
include(scrdir("simulation.jl"))

>Run | ⬆Above | ⬆Below | ⚙Debug
# ## Параметры

V = [10.0, 10.0]
c_a = 1.0
c_d = 1.0

>Run | ⬆Above | ⬆Below | ⚙Debug
# ## Базовая модель 2x2

A2, D2 = build_payoff_matrices(V, c_a, c_d)
eq2 = mixed_nash_2x2(A2, D2)

println("=== Базовая модель 2x2 ===")
println("Матрица A:\n", A2)
println("Матрица D:\n", D2)
println("Тип равновесия: ", eq2.type)
println("p = ", round(eq2.p, digits=3), " q = ", round(eq2.q, digits=3))

if eq2.type == "pure"
    i = argmax(eq2.p); j = argmax(eq2.q)

```

Рисунок 4.8: Скрипт дополнительного задания

В результате выполнения скрипта `extra_tasks.jl` получаем сравнение платёжных матриц базовой и расширенной моделей (рис. 4.9).

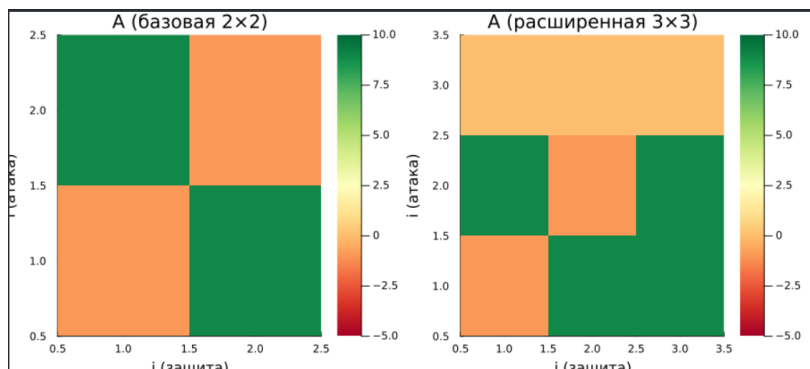


Рисунок 4.9: Сравнение платёжных матриц

4.6 Генерация производных форматов

Генерируем производные форматы с помощью скрипта `tangle.jl` (рис. 4.10).

```

Admin@DESKTOP-KPF3U04 MINGW64 ~/work/study/2026-1/2026-1==study--security-modeling/2026-1--study--se
curity-modeling/labs/lab04/project (master)
$ julia --project=. scripts/tangle.jl
[ Info: generating markdown page from 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-mode
ling\2026-1--study--security-modeling\labs\lab04\project\scripts\run_sims.jl'
[ Info: writing result to 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1-
study--security-modeling\labs\lab04\project\markdown\run_sims.qmd'
[ Info: generating notebook from 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\
2026-1--study--security-modeling\labs\lab04\project\scripts\run_sims.jl'
[ Info: writing result to 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1-
study--security-modeling\labs\lab04\project\notebooks\run_sims.ipynb'
[ Info: generating markdown page from 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-mode
ling\2026-1--study--security-modeling\labs\lab04\project\scripts\plot_results.jl'
[ Info: writing result to 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1-
study--security-modeling\labs\lab04\project\markdown\plot_results.qmd'
[ Info: generating notebook from 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\
2026-1--study--security-modeling\labs\lab04\project\scripts\plot_results.jl'
[ Info: writing result to 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1-
study--security-modeling\labs\lab04\project\notebooks\plot_results.ipynb'
[ Info: generating markdown page from 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-mode
ling\2026-1--study--security-modeling\labs\lab04\project\scripts\extra_tasks.jl'
[ Info: writing result to 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1-
study--security-modeling\labs\lab04\project\markdown\extra_tasks.qmd'
[ Info: generating notebook from 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\
2026-1--study--security-modeling\labs\lab04\project\scripts\extra_tasks.jl'
[ Info: writing result to 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1-
study--security-modeling\labs\lab04\project\notebooks\extra_tasks.ipynb'

```

Рисунок 4.10: Запуск генерации производных форматов

Получаем файлы других форматов: Jupyter Notebook, .qmd — Quarto Markdown (рис. 4.11).

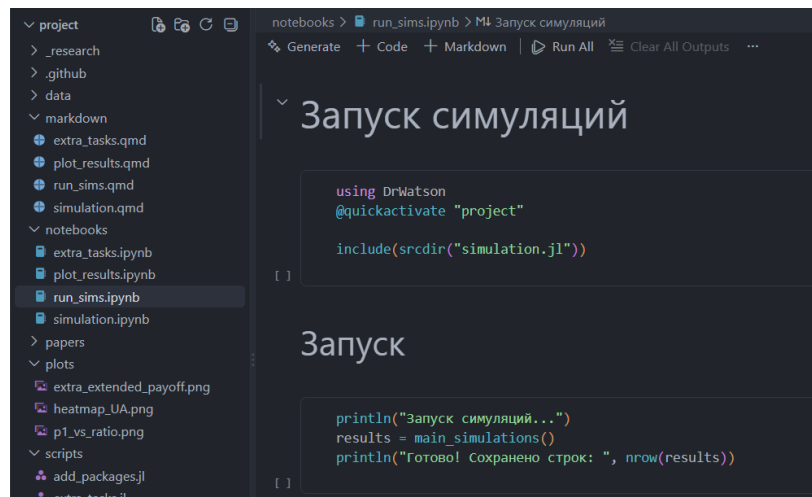


Рисунок 4.11: Полученные файлы других форматов

4.7 Реализация модуля simulation.jl

```

using LinearAlgebra
using DataFrames, CSV
using DrWatson
  
```

Строит платёжные матрицы A (атакующий) и D (защитник) для n объектов с ценностями V, стоимостями атаки c_a и защиты c_d. i — узел атаки, j — узел защиты: если $i \neq j$: атакующий получает $V[i] - c_a$, защитник теряет $V[i] + c_d$ если $i = j$: атака отражена, оба несут только затраты

```
function build_payoff_matrices(V::Vector{Float64}, c_a::Float64, c_d
    n = length(V)
    A = zeros(n, n)
    D = zeros(n, n)
    for i = 1:n, j = 1:n
        if i != j
            A[i, j] = V[i] - c_a
            D[i, j] = -V[i] - c_d
        else
            A[i, j] = -c_a
            D[i, j] = -c_d
        end
    end
    return A, D
end
```

Находит равновесие Нэша в игре 2×2 . Сначала проверяет чистые стратегии (седловая точка), затем вычисляет смешанные через условие безразличия.

```
function mixed_nash_2x2(A::Matrix{Float64}, D::Matrix{Float64})
    for i = 1:2, j = 1:2
        if A[i, j] >= A[3-i, j] && D[i, j] >= D[i, 3-j]
            p = zeros(2); p[i] = 1.0
            q = zeros(2); q[j] = 1.0
            return (p = p, q = q, type = "pure")
        end
    end
```

```

        end
    end
    denomA = (A[1,1] - A[2,1]) - (A[1,2] - A[2,2])
    q1 = abs(denomA) > 1e-10 ? clamp((A[2,2] - A[1,2]) / denomA, 0.0, 1.0)
    q = [q1, 1 - q1]
    denomD = (D[1,1] - D[1,2]) - (D[2,1] - D[2,2])
    p1 = abs(denomD) > 1e-10 ? clamp((D[2,2] - D[2,1]) / denomD, 0.0, 1.0)
    p = [p1, 1 - p1]
    return (p = p, q = q, type = "mixed")
end

```

Запускает один эксперимент для заданного набора параметров, возвращает словарь с результатами (стратегии, тип равновесия, выигрыши).

```

function run_simulation(params::Dict)
    V = params["V"]; c_a = params["c_a"]; c_d = params["c_d"]
    A, D = build_payoff_matrices(V, c_a, c_d)
    eq = mixed_nash_2x2(A, D)
    if eq.type == "pure"
        i = argmax(eq.p); j = argmax(eq.q)
        UA = A[i, j]; UD = D[i, j]
    else
        UA = eq.p' * A * eq.q; UD = eq.p' * D * eq.q
    end
    return Dict("p_1"=>eq.p[1], "p_2"=>eq.p[2], "q_1"=>eq.q[1], "q_2"=>eq.q[2],
        "type"=>eq.type, "UA"=>UA, "UD"=>UD, "V1"=>V[1], "V2"=>V[2],
        "c_a"=>c_a, "c_d"=>c_d)
end

```

Генерирует сетку параметров: 3 значения V1, 3 значения V2, 3 значения c_a, 3

значения $c_d \rightarrow 81$ комбинация.

```
function generate_params()
  dicts = []
  for v1 in [5.0, 10.0, 15.0], v2 in [5.0, 10.0, 15.0]
    for c_a in [0.0, 1.0, 3.0], c_d in [0.0, 1.0, 3.0]
      push!(dicts, Dict("V" => [v1, v2], "c_a" => c_a, "c_d" => c_d))
    end
  end
  return dicts
end
```

Прогоняет все комбинации параметров, собирает результаты в DataFrame и сохраняет в data/sims/results.csv.

```
function main_simulations()
  params_list = generate_params()
  rows = [run_simulation(p) for p in params_list]
  results = DataFrame(rows)
  mkpath(datadir("sims"))
  CSV.write(datadir("sims", "results.csv"), results)
  return results
end
```

Загружает сохранённые результаты из CSV.

```
function load_results()
  path = datadir("sims", "results.csv")
  isfile(path) ? CSV.read(path, DataFrame) : error("File not found. R
end
```

Расширенная матрица 3×3: добавляет стратегии «не атаковать» и «не защищать».

```
function build_payoff_matrices_3x3(V::Vector{Float64}, c_a::Float64,  
    A = zeros(3, 3)  
    D = zeros(3, 3)  
    for i = 1:2, j = 1:2  
        if i != j  
            A[i, j] = V[i] - c_a  
            D[i, j] = -V[i] - c_d  
        else  
            A[i, j] = -c_a  
            D[i, j] = -c_d  
        end  
    end  
    for j = 1:3  
        A[3, j] = 0.0  
        D[3, j] = j <= 2 ? -c_d : 0.0  
    end  
    for i = 1:2  
        A[i, 3] = V[i] - c_a  
        D[i, 3] = -V[i]  
    end  
    return A, D  
end
```

4.8 Запуск симуляций

```
using DrWatson
@quickactivate "project"

include(sourcedir("simulation.jl"))
```

```
println("XXXXXX XXXXXXXXXX...")
results = main_simulations()
println("XXXXXX! XXXXXXXXXX XXXXX: ", nrow(results))
```

4.9 Визуализация результатов

```
using DrWatson
@quickactivate "project"

using Plots, DataFrames, Statistics

include(sourcedir("simulation.jl"))
```

```
results = load_results()
```

```
filtered = results[(results.c_a .== 1.0) .& (results.c_d .== 1.0), :]
ratio = filtered.V1 ./ filtered.V2
scatter( ratio,
        filtered.p_1,
        group = filtered.type,
        xlabel = "V1 / V2",
```

```

        ylabel = "p1 (XXXXXXXXXX XXXX XX XXXXX 1)",
        title = "XXXXXXXXXX XXXXXXXXXXXX (c_a=1, c_d=1)",
        legend = :topright,
    )
    savefig(plotsdir("p1_vs_ratio.png"))

```

Тепловая карта выигрыша Нападающего

```

grp = groupby(filtered, [:V1, :V2])
summ = combine(grp, :UA => mean => :UA_mean)
heatmap(
    sort(unique(summ.V1)),
    sort(unique(summ.V2)),
    (x, y) -> summ[(summ.V1 .== x) .& (summ.V2 .== y), :UA_mean][1],
    xlabel = "V1",
    ylabel = "V2",
    title = "XXXXXXXXXX XXXXXXXX XXXXXXXXXXXX",
)
savefig(plotsdir("heatmap_UA.png"))

```

4.10 Дополнительное задание

Добавляем стратегию «Не атаковать» для Нападающего и «Не защищать ничего» для Защитника. Платёжная матрица становится 3×3. Модифицируем функции, находим равновесие и сравниваем с базовой моделью 2×2.

```
ENV["GKSwttype"] = "100"
```

```
using DrWatson
```

```
@quickactivate "project"
```

```
using Plots, LinearAlgebra  
include(sourcedir("simulation.jl"))
```

```
V = [10.0, 10.0]  
c_a = 1.0  
c_d = 1.0
```

```
A2, D2 = build_payoff_matrices(V, c_a, c_d)  
eq2 = mixed_nash_2x2(A2, D2)
```

```
println("=== 2x2 ===")  
println("A:\n", A2)  
println("D:\n", D2)  
println("Type: ", eq2.type)  
println("p = ", round(eq2.p, digits=3), " q = ", round(eq2.q, digits=3))
```

```
if eq2.type == "pure"  
    i = argmax(eq2.p); j = argmax(eq2.q)  
    UA2 = A2[i, j]; UD2 = D2[i, j]  
else  
    UA2 = eq2.p' * A2 * eq2.q  
    UD2 = eq2.p' * D2 * eq2.q  
end  
println("UA = ", round(UA2, digits=3), " UD = ", round(UD2, digits=3))
```

```

A3, D3 = build_payoff_matrices_3x3(V, c_a, c_d)

println("\n=== 3x3 ===")
println("A:\n", A3)
println("D:\n", D3)

p1 = heatmap(A2,
    title = "A (2x2)",
    xlabel = "j", ylabel = "i",
    color = :RdYlGn, clim = (-5, 10), size = (400, 350))

p2 = heatmap(A3,
    title = "A (3x3)",
    xlabel = "j", ylabel = "i",
    color = :RdYlGn, clim = (-5, 10), size = (400, 350))

plot(p1, p2, layout = (1, 2), size = (900, 400))
savefig(plotsdir("extra_extended_payoff.png"))
println("extra_extended_payoff.png")

```

5 Контрольные вопросы

1. Что такое антагонистическая игра? Почему модель «Защитник–Нападающий» можно считать игрой с нулевой суммой?

Антагонистическая игра — игра с прямо противоположными интересами игроков: выигрыш одного равен проигрышу другого.

Модель является игрой с нулевой суммой, так как сумма выигрышей постоянна:
- При успешной атаке: $(V_i - c_a) + (-V_i - c_d) = -c_a - c_d$ - При отражённой атаке: $(-c_a) + (-c_d) = -c_a - c_d$

Выигрыш Нападающего — это ущерб Защитника, что отражает антагонизм их интересов.

2. Дайте определение равновесия Нэша в чистых и смешанных стратегиях. Приведите пример, когда равновесия в чистых стратегиях не существует.

Равновесие Нэша — набор стратегий, при котором ни одному игроку не выгодно отклоняться в одностороннем порядке.

- В чистых стратегиях: каждый игрок выбирает одно действие с вероятностью 1.
- В смешанных стратегиях: игроки рандомизируют действия, выбирая вероятностное распределение.

Пример отсутствия чистого равновесия — игра «Орлянка». Ни одна из четырёх клеток не является равновесием, но существует смешанное равновесие (0.5, 0.5).

3. Как изменится равновесная стратегия защитника, если ущерб от успешной атаки на незащищённый сервер станет бесконечно большим?

При $V_1 \rightarrow \infty$: - Вероятность защиты первого актива $q_1^* \rightarrow 1$ - Защитник всегда защищает критически важный актив - Нападающий вынужден атаковать именно этот актив ($p_1^* \rightarrow 1$)

Практический смысл: критически важные активы (банковские БД, системы управления АЭС) должны защищаться всегда, независимо от затрат.

4. Зачем в информационной безопасности применяют смешанные стратегии? Приведите реальные примеры рандомизации в защите.

Причины применения:

1. Непредсказуемость — противник не может изучить паттерны.
2. Минимизация максимального ущерба — гарантированный уровень безопасности.
3. Оптимальное распределение ресурсов — «размазывание» риска.

Реальные примеры: - Honeypot — случайное развёртывание ложных сервисов-ловушек. - ASLR — случайное размещение кода в памяти ОС. - Рандомизированные аудиты — внезапные проверки безопасности. - Ротация портов — динамическое перераспределение сетевых сервисов. - Случайное сканирование — непредсказуемое время запуска сканеров уязвимостей.

6 Выводы

В результате лабораторной работы освоено применение теории игр для анализа противостояния в сфере информационной безопасности. Построена матричная игра, найдено равновесие Нэша в чистых и смешанных стратегиях, а также реализована визуализация зависимости ожидаемого выигрыша и равновесных вероятностей от стоимости защиты и величины ущерба.

Список литературы

1. DrWatson.jl Documentation. — URL: <https://juliadynamics.github.io/DrWatson.jl/stable/> (дата обр. 29.08.2026).
2. JuliaLang. — URL: <https://julialang.org/> (дата обр. 29.08.2026).
3. Literate.jl Documentation. — URL: <https://fredrikekre.github.io/Literate.jl/v2/> (дата обр. 29.08.2026).